

Pertemuan 10: Autentikasi JWT dan Data per Pengguna

Durasi	Level	Bentuk kerja	Prasyarat
150 menit	Lanjut	Individu	Pertemuan 9 (API /kegiatan dan /laporan)

Setelah praktikum ini, mahasiswa mampu:

- Hasil pertemuan 9 (atau snapshot `pert9/tugas9`).
- Paket Python baru: `PyJWT`, `pwdlib[argon2]`, `python-multipart`, `email-validator`.
- Paket Flutter baru: `flutter_secure_storage`.

Halaman 1

Penyimpanan token di HP. `shared_preferences` menyimpan teks biasa yang mudah dibaca pada perangkat yang di-*root*. Token adalah kunci akses akun, jadi simpan di `flutter_secure_storage`, yang memakai Keystore (Android) dan Keychain (iOS).

4. Langkah Praktikum

Bagian A: Dependensi dan Kunci Rahasia

Tambahkan ke `requirements.txt`:

```
PyJWT==2.15.1
pwnlib[argon2]==0.3.1
python-multipart==0.0.32
email-validator==2.3.0
```

Tambahkan ke `.env.example` (dan `.env`):

```
# Kunci rahasia untuk menandatangani JWT. Buat sendiri, misalnya:
# python3 -c "import secrets; print(secrets.token_hex(32))"
JWT_SECRET=ganti_dengan_kunci_acak_minimal_32_karakter
```

Teruskan ke layanan api di `docker-compose.yml` (`JWT_SECRET: ${JWT_SECRET}`). Tambahkan ke `Settings`:

```
# JWT: kunci rahasia WAJIB diganti lewat .env (JWT_SECRET).
jwt_secret: str = "kunci-pengembangan-jangan-dipakai-di-produksi"
jwt_algoritma: str = "HS256"
access_token_menit: int = 60
```

Bagian B: Model User dan Migrasi 0002

Tambahkan model User dan relasinya di `models.py`:

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    nama: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(150), unique=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    target_harian: Mapped[int] = mapped_column(default=20000)
    aktif: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())

    kegiatan: Mapped[list["Kegiatan"]] = relationship(
        back_populates="user", cascade="all, delete-orphan"
    )
```

Pada Kegiatan, tambahkan:

```
user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"), index=True)
...
user: Mapped[User] = relationship(back_populates="kegiatan")
```

Revisi `alembic/versions/0002_users_dan_pemilik_kegiatan.py`:

```
revision = "0002"
down_revision = "0001"

def upgrade() -> None:
    op.create_table(
        "users",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("nama", sa.String(100), nullable=False),
        sa.Column("email", sa.String(150), nullable=False, unique=True),
```

```

sa.Column("password_hash", sa.String(255), nullable=False),
sa.Column("target_harian", sa.Integer(), nullable=False, server_default="20000"),
sa.Column("aktif", sa.Boolean(), nullable=False, server_default=sa.true()),
sa.Column("created_at", sa.DateTime(), nullable=False, server_default=sa.func.now()),
)

# Kegiatan dari pertemuan 9 belum punya pemilik. Karena hanya data uji,
# data tersebut dihapus agar kolom user_id dapat dibuat NOT NULL.
op.execute("DELETE FROM kegiatan")
op.add_column("kegiatan", sa.Column("user_id", sa.Integer(), nullable=False))
op.create_index("ix_kegiatan_user_id", "kegiatan", ["user_id"])
op.create_foreign_key(
    "fk_kegiatan_user", "kegiatan", "users", ["user_id"], ["id"], ondelete="CASCADE"
)

```

Migrasi data. Menambah kolom NOT NULL ke tabel yang sudah berisi data harus menentukan nilai untuk baris lama. Di sini data uji dihapus. Pada aplikasi yang sudah dipakai, pilihannya adalah membuat kolom *nullable* dulu, mengisi nilainya, baru mengubahnya menjadi NOT NULL. Pola migrasi data seperti ini dipakai lagi di pertemuan 12.

Bagian C: Hash Password dan Token

app/security.py:

```

from datetime import datetime, timedelta, timezone

import jwt
from pwdlib import PasswordHash

from .config import settings

# Argon2: algoritma hash password yang direkomendasikan saat ini.
_hasher = PasswordHash.recommended()

def hash_password(password: str) -> str:
    return _hasher.hash(password)

def cek_password(password: str, hash_tersimpan: str) -> bool:
    return _hasher.verify(password, hash_tersimpan)

def buat_access_token(user_id: int) -> str:
    sekarang = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id), # subject: pemilik token
        "iat": sekarang, # issued at
        "exp": sekarang + timedelta(minutes=settings.access_token_menit),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm=settings.jwt_algoritma)

def baca_token(token: str) -> dict:
    """Melempar jwt.InvalidTokenError bila token rusak, palsu, atau kedaluwarsa."""
    return jwt.decode(token, settings.jwt_secret, algorithms=[settings.jwt_algoritma])

```

Bagian D: Dependency Pengguna yang Login

app/deps.py. *Dependency* `get_current_user` dapat dipasang di endpoint mana saja yang wajib login:

```

# tokenUrl dipakai tombol "Authorize" di Swagger UI (/docs).
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User:

```

```

"""Dependency: membaca header `Authorization: Bearer <token>`."""
gagal = HTTPException(
    status.HTTP_401_UNAUTHORIZED,
    "Token tidak valid atau sudah kedaluwarsa",
    headers={"WWW-Authenticate": "Bearer"},
)
try:
    payload = baca_token(token)
    user_id = int(payload["sub"])
except (jwt.InvalidTokenError, KeyError, ValueError):
    raise gagal from None
user = db.get(User, user_id)
if user is None:
    raise gagal
if not user.aktif:
    raise HTTPException(status.HTTP_403_FORBIDDEN, "Akun dinonaktifkan oleh admin")
return user

# Alias agar parameter endpoint lebih ringkas:
# def endpoint(db: DbSession, user: UserAktif): ...
DbSession = Annotated[Session, Depends(get_db)]
UserAktif = Annotated[User, Depends(get_current_user)]

```

Kolom aktif disiapkan sekarang dan akan dipakai admin untuk memblokir akun di pertemuan 12.

Bagian E: Endpoint Register, Login, dan Profil

Skema di `schemas.py` (impor juga `EmailStr`):

```

class UserCreate(BaseModel):
    nama: str = Field(min_length=1, max_length=100, examples=["Budi"])
    email: EmailStr = Field(examples=["budi@contoh.id"])
    password: str = Field(min_length=8, max_length=72, examples=["rahasia123"])

class UserOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    nama: str
    email: str
    target_harian: int

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

```

UserOut sengaja tidak memuat `password_hash`. Buat `app/routers/auth.py`:

```

@router.post("/auth/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(data: UserCreate, db: DbSession):
    email = data.email.lower()
    if db.scalar(select(User).where(User.email == email)):
        raise HTTPException(status.HTTP_409_CONFLICT, "Email sudah terdaftar")
    user = User(nama=data.nama.strip(), email=email, password_hash=hash_password(data.password))
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.post("/auth/login", response_model=Token)
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()], db: DbSession):
    """Login memakai form `username` (berisi email) dan `password`."""
    user = db.scalar(select(User).where(User.email == form.username.lower()))
    # Pesan sengaja sama untuk email salah maupun password salah.
    if user is None or not cek_password(form.password, user.password_hash):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Email atau password salah")
    if not user.aktif:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Akun dinonaktifkan oleh admin")

```

```

        return Token(access_token=buat_access_token(user.id))

@router.get("/users/me", response_model=UserOut)
def profil_saya(user: UserAktif):
    return user

```

Login memakai *form* (bukan JSON) mengikuti standar OAuth2 agar tombol **Authorize** di Swagger berfungsi. Paket `python-multipart` dibutuhkan untuk membaca form.

Bagian F: Melindungi Data Kegiatan dan Laporan

Ubah semua endpoint di `kegiatan.py` dan `laporan.py`:

```

def _ambil_atau_404(db: Session, kegiatan_id: int, user: User) -> Kegiatan:
    """Kegiatan milik user lain dianggap tidak ada (404, bukan 403),
    agar orang lain tidak bisa menebak id mana yang ada."""
    k = db.get(Kegiatan, kegiatan_id)
    if k is None or k.user_id != user.id:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Kegiatan tidak ditemukan")
    return k

@router.get("", response_model=list[KegiatanOut])
def daftar_kegiatan(response: Response, db: DbSession, user: UserAktif, ...):
    stmt = select(Kegiatan).where(Kegiatan.user_id == user.id)
    ...

@router.post("", response_model=KegiatanOut, status_code=status.HTTP_201_CREATED)
def tambah_kegiatan(data: KegiatanInput, db: DbSession, user: UserAktif):
    k = Kegiatan(**data.model_dump(), user_id=user.id)
    ...

```

Di `laporan.py`, tambahkan `Kegiatan.user_id == user.id` pada setiap `where`. Daftarkan `auth.router` di `main.py`.

Urutan parameter Python. Parameter tanpa nilai bawaan (`db: DbSession, user: UserAktif`) harus ditulis sebelum parameter yang memiliki nilai bawaan (`tanggal: date | None = None`).

Bagian G: Menguji Otorisasi

Jalankan `docker compose up -d --build` (migrasi 0002 berjalan otomatis), lalu uji di Swagger memakai tombol **Authorize** (isi *username* dengan email):

No	Langkah	Hasil yang diharapkan
1	POST <code>/auth/register</code> Budi, lalu Budi lagi	201, lalu 409 Email sudah terdaftar
2	Register dengan email bukan-email atau password 123	422
3	Login dengan password salah	401 Email atau password salah
4	GET <code>/kegiatan</code> tanpa Authorize	401 Not authenticated
5	Login Budi, Authorize, POST <code>/kegiatan</code> , GET <code>/users/me</code>	201; profil Budi tanpa password_hash
6	Register dan login Ani, GET <code>/kegiatan</code>	[] (data Budi tidak terlihat)
7	Sebagai Ani, DELETE <code>/kegiatan/{id milik Budi}</code>	404 dan data Budi tetap ada
8	Ubah satu huruf token lalu panggil <code>/users/me</code>	401

Buka token di jwt.io (gunakan data uji saja): payload sub dan exp terbaca, tetapi tanda tangan tidak valid tanpa JWT_SECRET.

Checkpoint G: delapan skenario sesuai. Di Adminer, kolom password_hash berawalan \$argon2id\$.

Bagian H: Login di Flutter

1. Tambahkan flutter_secure_storage: ^11.2.0 di pubspec.yaml.
2. Buat model lib/models/pengguna.dart (id, nama, email, targetHarian, beserta fromJson/toJson).
3. Buat abstraksi penyimpanan token di lib/services/token_storage.dart agar test tidak membutuhkan plugin native:

```
abstract class TokenStorage {
  Future<String?> baca(String kunci);
  Future<void> tulis(String kunci, String nilai);
  Future<void> hapusSemua();
}

// Disimpan terenkripsi: Keystore (Android) dan Keychain (iOS).
class SecureTokenStorage implements TokenStorage {
  final _storage = const FlutterSecureStorage();

  @override
  Future<String?> baca(String kunci) => _storage.read(key: kunci);

  @override
  Future<void> tulis(String kunci, String nilai) =>
    _storage.write(key: kunci, value: nilai);

  @override
  Future<void> hapusSemua() => _storage.deleteAll();
}
```

Buat juga MemoryTokenStorage (berbasis Map) untuk pengujian.

4. Status login global di lib/state/sesi.dart. Polanya sama dengan versiData pertemuan 7, yaitu ValueNotifier:

```
/// Pengguna yang sedang login; null berarti belum login.
final pengguna = ValueNotifier<Pengguna?>(null);

/// Access token di memori agar tidak membaca storage di setiap request.
String? accessToken;

TokenStorage penyimpananToken = SecureTokenStorage();

Future<void> muatSesi() async { /* baca token + profil dari storage */ }
Future<void> simpanSesi(String token, Pengguna p) async { /* tulis lalu set pengguna */ }
Future<void> hapusSesi() async { /* hapus semua lalu pengguna.value = null */ }
```

5. Buat lib/services/api_client.dart, sebuah http.Client pembungkus (pola decorator) yang menambahkan header token:

```
class ApiClient extends http.BaseClient {
  final http.Client _inner;

  ApiClient([http.Client? inner]) : _inner = inner ?? http.Client();

  @override
  Future<http.StreamedResponse> send(http.BaseRequest request) {
    final token = accessToken;
    if (token != null) {
      request.headers['Authorization'] = 'Bearer $token';
    }
  }
}
```

```

    return _inner.send(request);
  }

  @override
  void close() => _inner.close();
}

```

Ubah nilai bawaan client di KegiatanApi dan LaporanApi menjadi client ?? ApiClient().

6. Buat lib/services/auth_api.dart dengan daftar(nama, email, password), masuk(email, password), dan profil(token). Login mengirim form: bila body berupa Map<String, String>, paket http otomatis memakai application/x-www-form-urlencoded:

```

Future<Pengguna> masuk(String email, String password) async {
  final res = await _client
    .post(_uri('/auth/login'), body: {'username': email, 'password': password})
    .timeout(batasWaktu);
  if (res.statusCode != 200) throw ApiException.dariRespons(res);
  final token =
    (jsonDecode(res.body) as Map<String, dynamic>)['access_token'] as String;

  final p = await profil(token);
  await simpanSesi(token, p);
  return p;
}

```

7. Buat LoginPage (email, password dengan tombol tampilkan/sembunyikan, tombol **Masuk** dengan indikator loading, tautan **Daftar** dan **Status server**) dan RegisterPage (nama, email, password minimal 8, ulangi password; setelah berhasil langsung login).
8. Pilih halaman awal berdasarkan sesi di main.dart. Panggil await muatSesi(); di main() setelah muatPengaturan():

```

'/': (_) => ValueListenableBuilder<Pengguna?>(
  valueListenable: pengguna,
  builder: (context, p, _) =>
    p == null ? const LoginPage() : const ShellPage(),
),
'/daftar': (_) => const RegisterPage(),

```

9. Di halaman Pengaturan, tampilkan kartu akun (nama, email) dengan tombol **Keluar** dan dialog konfirmasi yang memanggil hapusSesi(). Tambahkan pengguna ke Listenable.merge.
10. Uji di test/auth_api_test.dart dengan MemoryTokenStorage dan MockClient: login mengirim form dan menyimpan sesi, login salah tidak menyimpan sesi, sesi dipulihkan dari storage, ApiClient menambah header, dan hapusSesi mengosongkan semuanya.

Checkpoint H: aplikasi dibuka di halaman Login; daftar akun baru langsung masuk ke Hari Ini; tutup lalu buka aplikasi masih dalam keadaan login; **Keluar** kembali ke Login. Pada pertemuan ini data kegiatan di UI masih dari SQLite. Integrasi dengan API dilakukan di pertemuan 11.

5. Latihan Mandiri

1. Tambahkan endpoint POST /auth/ganti-password (password lama + baru) yang wajib login.
2. Ubah ACCESS_TOKEN_MENIT menjadi 1, login, tunggu 1 menit, lalu panggil /users/me. Catat pesan yang muncul.
3. Tambahkan validasi password: minimal satu huruf dan satu angka, baik di Pydantic maupun di form Flutter.

4. Tampilkan inisial nama pengguna di AppBar halaman Hari Ini.

6. Tugas

Access token berumur panjang berbahaya bila dicuri, sedangkan umur pendek membuat pengguna sering login ulang. Solusinya adalah **refresh token**:

- Backend: access token 15 menit dan refresh token 7 hari (atur lewat `.env`). Payload memuat `"jenis": "access" atau "refresh".get_current_user` menolak refresh token. Login membalas `access_token + refresh_token`. Endpoint POST `/auth/refresh` `{"refresh_token": "..."} membalas pasangan token baru, dan menolak access token serta akun nonaktif.`
- Flutter `ApiClient`: bila respons **401**, minta token baru ke `/auth/refresh`, simpan, lalu **ulangi request sekali**. Bila refresh ditolak, panggil `hapusSesi()` agar aplikasi kembali ke Login (*auto-logout*). Bila server tidak dapat dihubungi saat refresh, **jangan** logout.
- Beberapa request yang gagal bersamaan (misalnya tiga request halaman Laporan) hanya boleh memicu **satu** refresh. Petunjuk: simpan `Future` refresh yang sedang berjalan di variabel `static`.
- 401 dari `/auth/login` tidak boleh memicu refresh.
- Unit test untuk setiap perilaku di atas.
- Bukti isolasi data: skrip `curl` atau tangkapan layar Swagger yang menunjukkan pengguna A tidak dapat membaca, mengubah, maupun menghapus kegiatan pengguna B.

7. Rubrik Penilaian

Komponen	Bobot
Backend: register, login, <code>/users/me</code> , hash Argon2, isolasi data (bagian A–G)	30%
Flutter: secure storage, <code>ApiClient</code> , Login/Register, keluar (bagian H)	20%
Latihan mandiri	10%
Tugas: refresh token di backend	15%
Tugas: auto-refresh, auto-logout, satu refresh bersamaan, beserta test	25%

8. Pertanyaan Refleksi

1. Mengapa pesan login salah dibuat sama untuk email salah dan password salah?
2. Isi payload JWT dapat dibaca siapa saja. Lalu apa yang membuatnya aman?
3. Apa kelemahan JWT *stateless* bila admin ingin mencabut akses seseorang **seketika**? Bagaimana kolom `aktif` membantu?
4. Mengapa mengakses data orang lain dibalas 404 dan bukan 403?
5. Mengapa token tidak disimpan di `shared_preferences`?

9. Troubleshooting Umum

Masalah	Solusi
Form data requires "python-multipart"	Tambahkan <code>python-multipart</code> ke <code>requirements.txt</code> , lalu <code>docker compose up -d --build</code>
email-validator is not installed	Tambahkan <code>email-validator</code> , lalu build ulang
Semua request 401 setelah <code>.env</code> diubah	<code>JWT_SECRET</code> berganti sehingga token lama tidak valid. Login ulang
Parameter without a default cannot follow a parameter with a default	Pindahkan <code>db: DbSession</code> dan <code>user: UserAktif</code> ke depan parameter ber- <i>default</i>
Migrasi 0002 gagal Cannot add foreign key constraint	Masih ada kegiatan tanpa pemilik. Pastikan <code>DELETE FROM kegiatan</code> dijalankan sebelum <code>add_column</code>
Flutter: login berhasil tetapi tetap di halaman Login	Pastikan <code>simpanSesi</code> mengisi <code>pengguna.value</code> dan rute <code>'/'</code> memakai <code>ValueListenableBuilder</code>
<code>MissingPluginException</code> dari secure storage	Hentikan aplikasi lalu <code>flutter run</code> ulang setelah menambah paket. Di test, pakai <code>MemoryTokenStorage</code>

10. Referensi

- FastAPI, *OAuth2 with Password (and hashing), Bearer with JWT tokens*: fastapi.tiangolo.com/tutorial/security/oauth2-jwt
- RFC 7519 JSON Web Token; debugger: jwt.io
- OWASP Password Storage Cheat Sheet: cheatsheetseries.owasp.org
- `flutter_secure_storage`: pub.dev/packages/flutter_secure_storage